Samuel Umoren, Benjamin Diaz
CECS 327 – Assignment 8 Report
Date: 4/30/2026

Build a Distributed End-to-End IoT System IoT / Distributed Systems Report

We made a distributed end-to-end IoT system for this project that used our TCP client/server from Assignment 6, our DataNiz and NeonDB setup from Assignment 7, and a shared data model that worked in two different student environments. Everyone on the team had their own smart home, DataNiz account, virtual IoT devices, and database. The goal was to make the system work like a real distributed environment, where both houses collect their own sensor data, share some of it with each other, and still be able to tell which house each record came from.

The DataNiz virtual devices sent IoT data through broker topics into NeonDB, and the TCP client and server took care of the user queries. The client was changed so that it only accepts the three exact queries that are needed for the assignment. Any other input is rejected with a message that is easy to understand. The main job on the server side was to use data from both smart-house environments to process the supported queries. This meant that the server had to decide if the local database had enough information to answer a question or if it had to get missing historical data from the teammate's original database. That part was important because the assignment says that the system has to work in a distributed way and not just by putting everything into one central database.

The DataNiz sharing feature was a big part of the project. When sharing was turned on, each teammate's database started getting data from the other person's shared device. The most important thing to know is that the sharing is one-way, so data from before sharing started doesn't get copied over. Because of this, each database only has a small part of the full team dataset. Everyone always has all of their own historical data, but they only have their teammates' shared data from the time sharing started and after. That turned out to be one of the project's main ideas for distributed systems. It meant that our server logic had to know about incomplete time windows and possibly missing peer history.

We used a lot of metadata to help us keep things in order. DataNiz's metadata helped us figure out which records belonged to House A or House B, which device made the data, and whether it was local or shared. NeonDB is one of the best examples of this. The topic column had records from both student environments. Even though the records from both houses were in the same table, we could still tell which house created each row by looking at the ownership of the topic and the information about the related device. Metadata also helped us find the right records for each query, put timestamps in the right order, and make sure the server combined results in a way that still respected house ownership.

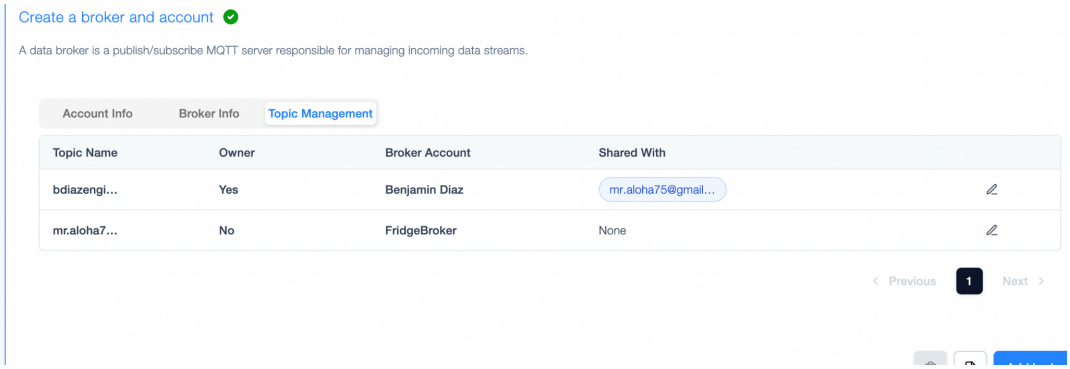
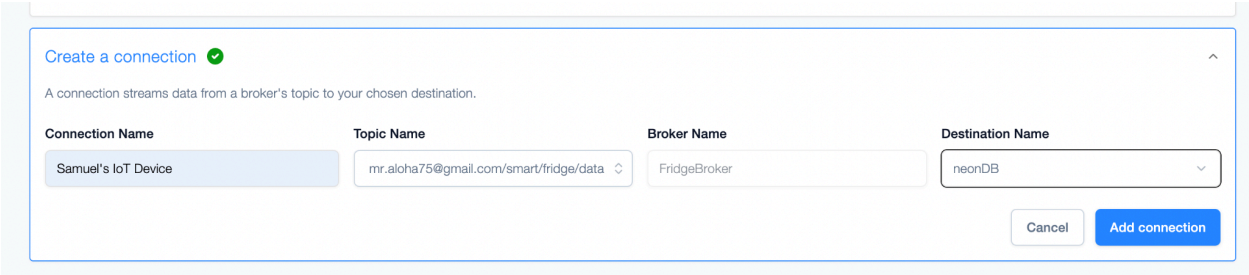
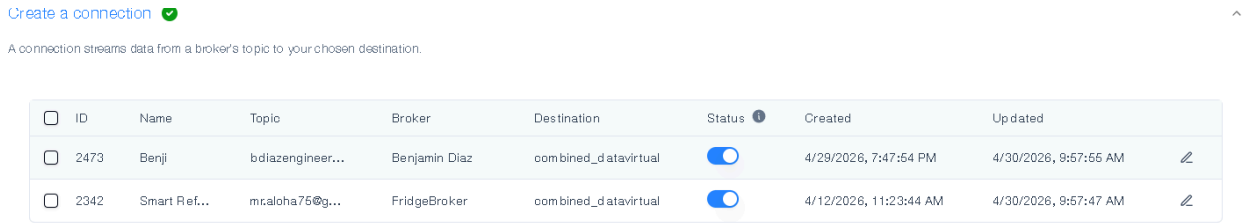
The system was set up to answer three set questions: how much water smart dishwashers use on average per cycle, how much moisture is in kitchen fridges on average, and which house used

more electricity in the last 24 hours. The moisture question used data from sensors in the refrigerator, the water question used data from the dishwasher cycle, and the electricity question looked at energy-related activity for each house separately. The server had to figure out how much electricity each house used, compare the totals, and tell which house used more and by how much. The assignment said that results had to be shown in PST and imperial units when necessary, so the server design also had to include converting timestamps and units when necessary. We wrote down any assumptions we made for those conversions as part of the project design.

One thing that stood out during the project was that DataNiz was helpful for building the pipeline. However, it wasn't always clear if the data was only being created on the visualization page or if it was actually being written to the Neon database. Another problem was working around forward-only replication, which meant that shared peer history might not be available for earlier time windows. That meant that instead of assuming that every query could be answered from one local table, we had to think more carefully about distributed completeness. Adding a live pipeline status feature that shows in real time whether data is being created, sent through the broker, and successfully written into the destination database would be a big step forward for DataNiz. That would make it much easier to find and fix bugs, and it would make the platform more useful for projects like this.

Samuel Umoren, Benjamin Diaz
CECS 327 – Assignment 8 Report
Date: 4/30/2026

Images

payload json	topic varchar(255)	time timestamp with time z...	createdAt timestamp with...	updatedAt timestamp
{"timestamp":"17775811…	mr.aloha75@gmail.com/s…	2026-04-30 20:32:15+00	2026-04-30 20:32:15.70…	2026-04-30 20:32
{"timestamp":"17775811…	bdiazengineer@gmail.co…	2026-04-30 20:32:14+00	2026-04-30 20:32:14.69…	2026-04-30 20:32
{"timestamp":"17775811…	mr.aloha75@gmail.com/s…	2026-04-30 20:32:12+00	2026-04-30 20:32:12.81…	2026-04-30 20:32
{"timestamp":"17775811…	bdiazengineer@gmail.co…	2026-04-30 20:32:04+00	2026-04-30 20:32:05.29…	2026-04-30 20:32
{"timestamp":"17775811…	bdiazengineer@gmail.co…	2026-04-30 20:32:04+00	2026-04-30 20:32:04.28…	2026-04-30 20:32
{"timestamp":"17775811…	mr.aloha75@gmail.com/s…	2026-04-30 20:32:02+00	2026-04-30 20:32:02.65…	2026-04-30 20:32
{"timestamp":"17775810…	mr.aloha75@gmail.com/s…	2026-04-30 20:31:16+00	2026-04-30 20:31:16.36…	2026-04-30 20:31
{"timestamp":"17775810…	bdiazengineer@gmail.co…	2026-04-30 20:31:15+00	2026-04-30 20:31:15.34…	2026-04-30 20:31
{"timestamp":"17775810…	mr.aloha75@gmail.com/s…	2026-04-30 20:31:13+00	2026-04-30 20:31:13.37…	2026-04-30 20:31
{"timestamp":"17775810…	bdiazengineer@gmail.co…	2026-04-30 20:31:05+00	2026-04-30 20:31:05.94…	2026-04-30 20:31
{"timestamp":"17775810…	bdiazengineer@gmail.co…	2026-04-30 20:31:04+00	2026-04-30 20:31:04.84…	2026-04-30 20:31
{"timestamp":"17775810…	mr.aloha75@gmail.com/s…	2026-04-30 20:31:03+00	2026-04-30 20:31:03.22…	2026-04-30 20:31
{"timestamp":"17775810…	mr.aloha75@gmail.com/s…	2026-04-30 20:30:16+00	2026-04-30 20:30:16.95…	2026-04-30 20:30
{"timestamp":"17775810…	bdiazengineer@gmail.co…	2026-04-30 20:30:15+00	2026-04-30 20:30:15.94…	2026-04-30 20:30
{"timestamp":"17775810…	mr.aloha75@gmail.com/s…	2026-04-30 20:30:13+00	2026-04-30 20:30:14.02…	2026-04-30 20:30
{"timestamp":"17775810…	bdiazengineer@gmail.co…	2026-04-30 20:30:06+00	2026-04-30 20:30:06.44…	2026-04-30 20:30
{"timestamp":"17775810…	bdiazengineer@gmail.co…	2026-04-30 20:30:05+00	2026-04-30 20:30:05.76…	2026-04-30 20:30
{"timestamp":"17775810…	mr.aloha75@gmail.com/s…	2026-04-30 20:30:03+00	2026-04-30 20:30:03.66…	2026-04-30 20:30
{"timestamp":"17775809…	mr.aloha75@gmail.com/s…	2026-04-30 20:29:15+00	2026-04-30 20:29:15.75…	2026-04-30 20:29
{"timestamp":"17775809…	bdiazengineer@gmail.co…	2026-04-30 20:29:14+00	2026-04-30 20:29:14.72…	2026-04-30 20:29
{"timestamp":"17775809…	mr.aloha75@gmail.com/s…	2026-04-30 20:29:12+00	2026-04-30 20:29:12.85…	2026-04-30 20:29
{"timestamp":"17775809…	bdiazengineer@gmail.co…	2026-04-30 20:29:04+00	2026-04-30 20:29:04.36…	2026-04-30 20:29
{"timestamp":"17775809…	bdiazengineer@gmail.co…	2026-04-30 20:29:04+00	2026-04-30 20:29:05.47…	2026-04-30 20:29
{"timestamp":"17775809…	mr.aloha75@gmail.com/s…	2026-04-30 20:29:02+00	2026-04-30 20:29:02.67…	2026-04-30 20:29
{"timestamp":"17775808…	mr.aloha75@gmail.com/s…	2026-04-30 20:28:16+00	2026-04-30 20:28:16.33…	2026-04-30 20:28
{"timestamp":"17775808…	bdiazengineer@gmail.co…	2026-04-30 20:28:15+00	2026-04-30 20:28:15.36…	2026-04-30 20:28
{"timestamp":"17775808…	mr.aloha75@gmail.com/s…	2026-04-30 20:28:13+00	2026-04-30 20:28:13.41…	2026-04-30 20:28
{"timestamp":"17775808…	bdiazengineer@gmail.co…	2026-04-30 20:28:05+00	2026-04-30 20:28:06.13…	2026-04-30 20:28

Samuel Umoren, Benjamin Diaz
CECS 327 – Assignment 8 Report
Date: 4/30/2026